

Hw8_q1

Size	Result	Spend time
100	372921	1.4007091522216797e-06
1,000	782575	2.4996697902679443e-06
10,000	857407	1.7993152141571045e-06
100,000	250709	2.000480890274048e-06
1,000,000	155203	1.80024653673172e-06
100,000,000	694763	0.0022175004705786705

در این برنامه، صرفاً عنصر اول یک لیست به عنوان خروجی نمایش داده می‌شود. بنابراین یک عملیات $\text{return } l[0]$ داریم، که پیچیدگی زمانی آن $O(1)$ یا به عبارتی $O(c)$ می‌باشد. دلیل افزایش زمان سپری شده برای لیست با طول 100 میلیون اشغال زیاد cache، در CPU و Ram است. ولی در کل پیچیدگی زمانی ثابت است.

Hw8_q2

Number	Result	Spend time
16	4	2.50060111284256e-06
32	5	1.300126314163208e-06
64	6	8.996576070785522e-07
128	7	5.997717380523682e-07
256	8	6.994232535362244e-07
512	9	8.00006091594696e-07
1024	10	1.000240445137024e-06
2^{20}	20	1.600012183189392e-06
2^{40}	40	4.299916326999664e-06
2^{60}	60	6.1998143792152405e-06
2^{80}	80	9.000301361083984e-06
2^{100}	100	1.1999160051345825e-05

در داده های کوچک رشد زمانی ثابت به نظر می‌رسد. اما با افزایش عدد، رابطه ای کشف می‌شود. چون در حلقه عدد مورد نظر هربار نصف می‌شود، پیچیدگی زمانی $O(\log_2 n)$ است. که به بیان دیگر $O(\log n)$ می‌باشد.

Hw8_q3

Size	Result	Spend time
10,000	999929	0.0007974999025464058
20,000	999932	0.0015481999143958092
40,000	999966	0.0031783003360033035
80,000	999995	0.004632500000298023
160,000	999980	0.012446499429643154
320,000	999986	0.023325499147176743
640,000	999998	0.04612009972333908
1,280,000	999998	0.06486769951879978

زمان سپری شده وابسته به طول لیست است. از طرفی به ازای دو برابر شدن طول لیست، مدت زمان سپری شده تقریباً دو برابر می‌شود. پس نتیجه میگیریم پیچیدگی زمانی $O(n)$ است.

Hw8_q4

Size	Result	Spend time
500	13	0.01869689952582121
1,000	57	0.07499559968709946
2,000	202	0.3038198994472623
4,000	767	1.08223420009017
8,000	3240	4.555437200702727
16,000	12711	17.176930299960077

چون برنامه با کمک دو حلقه که هر دو به طول لیست وابسته اند، پیچیدگی زمانی اجرای برنامه $O(n^2)$ خواهد بود. یعنی با دو برابر شدن طول لیست، زمان سپری شده تقریباً چهار برابر می‌شود.